

Contents

1. Moodle Quiz
2. Häufige Fehler aus Exercise Sheet 1
3. Beschreibung von Exercise Sheet 2
4. Theory recap - Maximum Subarray Sum
5. In-Class Aufgaben

1. Moodle Quiz

Aufgabe 1

In the computation model used in the lecture, how do we measure the running time of an algorithm? Select one answer.

- ☒ a. By the number of elementary operations it performs.
- ☐ b. By the number of digits in its output.
- ☐ c. By the number of seconds it takes on one particular laptop.

Lösung:

Wir wollen die Komplexität eines Algorithmus unabhängig von der Maschine definieren, auf der er ausgeführt wird. Aus diesem Grund definieren wir die Komplexität als die Anzahl der elementaren Operationen, die der Algorithmus durchführt, um das Ergebnis zu berechnen. Diese elementare Operationen können Additionen, Multiplikationen, Speicherzugriffe, Vergleiche und andere sein.

=> Antwort a)

Aufgabe 2

In Karatsuba's algorithm as presented in the lecture, the smaller multiplication problems are solved using the same idea again, until only single-digit multiplications remain.

- ☒ True
- ☐ False

Lösung:

Der Karatsuba-Algorithmus ist ein rekursiver Algorithmus. Das bedeutet, dass wir die Lösungen der kleineren Probleme (in diesem Fall die Multiplikation von Zahlen mit weniger Ziffern) verwenden, um die größeren Probleme zu lösen. Es bedeutet auch, dass die Art und Weise, wie wir die kleineren Probleme lösen, dieselbe ist wie bei den größeren Problemen. Die einzige Ausnahme ist der Basisfall, der besonders ist, da die Rekursion in diesem Fall endet und er anders behandelt wird. Für diese Übung tritt der Basisfall ein, wenn die Zahlen nur ein Ziffer haben.
=> Die Aussage ist wahr (Antwort: **True**)

Aufgabe 3

What is the maximum subarray sum of $A = (4, -6, 3)$?

Answer:

Lösung:

Ein Subarray ist eine Teilmenge der Elemente des ursprünglichen Arrays, die sich an aufeinanderfolgenden Indizes befinden. Das bedeutet, dass wir die -6 nicht überspringen und die 4 und 3 nehmen können, um eine Subarray-Summe von 7 zu erhalten. Daher ist die Antwort die größere Zahl zwischen 4 und 3, nämlich **4**.

Aufgabe 4

$$1000n \leq O(n^2)$$

- ☒ True ☐
- ☒ False ☐

Lösung:

$$\lim_{n \rightarrow \infty} \frac{1000n}{n^2} = \lim_{n \rightarrow \infty} \frac{1000}{n} = 0 \Rightarrow 1000n \leq O(n^2) \text{ und die Aussage ist wahr}$$

$\Rightarrow$ Antwort: **True**

Aufgabe 5

Let $f(n) = 5n + 2$ for positive integers n . For each statement, decide whether it is true or false.

True	False		
☐	☐	$f \in \Theta(1).$	☒
☐	☐	$f \in \Theta(n).$	☒
☐	☐	$f \in \Theta(n^2).$	☒

Lösung

Thm. Sei N unendlich und $N \subseteq \mathbb{N}$ und seien $f: N \rightarrow \mathbb{R}^+, g: N \rightarrow \mathbb{R}^+$ zwei Funktionen

- Falls $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \in \mathbb{R}^+$, dann $f \in \Theta(g)$

$$\lim_{n \rightarrow \infty} \frac{5n+2}{1} = \lim_{n \rightarrow \infty} 5n+2 = \infty \Rightarrow f \notin \Theta(1) \Rightarrow \text{False}$$

$$\lim_{n \rightarrow \infty} \frac{5n+2}{n} = \lim_{n \rightarrow \infty} \frac{5 + \frac{2}{n}}{1} = 5 \Rightarrow f \in \Theta(n) \Rightarrow \text{True}$$

$$\lim_{n \rightarrow \infty} \frac{5n+2}{n^2} = \lim_{n \rightarrow \infty} \frac{\frac{5}{n} + \frac{2}{n^2}}{1} = 0 \Rightarrow f \notin \Theta(n^2) \Rightarrow \text{False}$$

Aufgabe 6

For any real numbers x and y , if $x \leq 5$ and $y \leq 5$, then $x \leq y$.

☐ True

☐ False

Lösung:

Wir zeigen, dass die Aussage falsch ist durch ein Gegenbeispiel.

Sei $x=4$ und $y=3$.

Es gilt $x \leq 5$ und $y \leq 5$, weil $4 \leq 5$ und $3 \leq 5$.

$x \leq y$ gilt aber nicht, weil $4 \geq 3$.

Aufgabe 7

Karatsuba's algorithm multiplies two n -digit numbers, where we assume for simplicity that $n \geq 2$ is a power of two. How many recursive multiplication calls on two $n/2$ -digit numbers does one step make directly? Select one answer.

- ☒ a. 2
- ☒ b. 3
- ☒ c. 4

Lösung:

Seien A und B die n -stellige Zahlen. Wir trennen A und wir kriegen 2 $n/2$ -stellige Zahlen $A = 10^{n/2}a_1 + a_0$
trennen B und wir kriegen 2 $n/2$ -stellige Zahlen $B = 10^{n/2}b_1 + b_0$.
 $A \cdot B = a_0b_0 + 10^{n/2}a_0b_1 + 10^{n/2}(a_1 - a_0)(b_1 - b_0) + 10^{n/2}a_1b_1 + 10^n a_1b_1$.
Wir multiplizieren nur a_0b_0 , $(a_1 - a_0)(b_1 - b_0)$ und a_1b_1 .

$\Rightarrow$ 3 Multiplikationen

Aufgabe 8

Let $S_k = \sum_{i=1}^k i^2$, where k is a positive integer. Which term must be added to S_k to obtain S_{k+1} ? Select one answer.

- ☒ a. $k+1$
- ☒ b. $(k+1)^2$
- ☒ c. k^2
- ☒ d. i^2
- ☒ e. $(i+1)^2$

Lösung:

$$S_k = \sum_{i=1}^k i^2 \quad \text{und} \quad S_{k+1} = \sum_{i=1}^{k+1} i^2$$

$$\Rightarrow S_{k+1} - S_k = \sum_{i=1}^{k+1} i^2 - \sum_{i=1}^k i^2 = (k+1)^2 \Rightarrow 6)(k+1)^2$$

Aufgabe 9

For a list of $n \geq 3$ distinct integers, a recursive call has found the maximum a' of $a_1, \dots, a_{n-1}$. Which comparison is always sufficient to determine the maximum of the whole list $a_1, \dots, a_n$? Select one answer.

- ☒ a. Compare a_1 with a_n .
- ☒ b. Compare a_{n-1} with a_n .
- ☒ c. Compare a' with a_n .

Lösung:

Wir wissen, dass a' das Maximum aus $a_1, \dots, a_{n-1}$ ist.

$$\Rightarrow a' \geq a_i \text{ für } \forall i \in [n-1]$$

$$\Rightarrow \text{falls } a_n \geq a', \text{ dann } a_n \geq a_i \text{ für } \forall i \in [n] \text{ und } \text{Max} = a_n,$$

$$\text{sonst } a_n < a', \text{ dann } a' \geq a_i \text{ für } \forall i \in [n] \text{ und } \text{Max} = a'$$

$\Rightarrow$ Antwort c)

Aufgabe 10

Let $A = (a_1, \dots, a_n)$ be an array of real numbers with $n \geq 5$. For each $j \in \{1, \dots, n\}$, let $S_j = \sum_{i=1}^j a_i$ be the sum of its first j entries. Which expression equals $a_3 + a_4 + a_5$? Select one answer.

- ☒ a. $S_5 - S_2$
- ☒ b. $S_5 - S_3$
- ☒ c. $S_5 + S_2$

Lösung:

$$S_5 = \underbrace{a_1 + a_2}_{S_2} + \underbrace{a_3 + a_4 + a_5}_{S_5 - S_2} \Rightarrow \text{Antwort a)}$$

Aufgabe 11

What is the maximum subarray sum of $A = (-3, -1, -4)$?

Answer:



Lösung:

Das leere Subarray hat die Summe 0. Da alle Elemente des Arrays negativ sind, hat jedes nicht-leere Subarray eine negative Summe seiner Elemente.

$\Rightarrow$ Antwort: 0

Aufgabe 12

For any odd integer $b \in \mathbb{N}_0$, the following holds:

If $f : \mathbb{N}_0 \rightarrow \mathbb{N}_0$ satisfies $f(n+1) = b \cdot f(n)$ for all $n \in \mathbb{N}_0$, then $f(k)$ is odd for all $k \in \mathbb{N}_0$.

Here $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.

☒ True ☒

☒ False ☒

Lösung:

Wir zeigen, dass die Aussage falsch ist mit einem Gegenbeispiel.

Sei $f(0) = 0$

$$\Rightarrow f(1) = b \cdot f(0) = b \cdot 0 = 0$$

$$\Rightarrow f(2) = 0 \Rightarrow f(n) = 0, \forall n \in \mathbb{N}_0$$

$\Rightarrow$ die Aussage ist falsch.

Aufgabe 13

For any two functions $f, g : \mathbb{N}_0 \rightarrow \mathbb{N}_0$, the following holds:

If $f(0) > g(0)$ and $f(n+1) - f(n) > g(n+1) - g(n)$ for all $n \in \mathbb{N}_0$, then $f(k) > g(k)$ for all $k \in \mathbb{N}_0$.

Here $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.

☒ True ☒

☒ False ☒

Lösung:

Die Aussage ist wahr (True) und wir können sie per Induktion beweisen.

2. Häufige Fehler aus Exercise Sheet 1

Ausdrücke: 5 , $a+b$, $\frac{n^2+3n+5}{4\sqrt{n}}$

Aussagen: $a \leq b$, $\sqrt{k} + 100k^2 \leq 101k^3$

Ausdrücke liefern keine Informationen, während Aussagen dies tun.

"=", " $\leq$ ", " $\geq$ ", "<", ">" sind Operatoren, die Ausdrücke verbinden. Die Verknüpfung zweier Ausdrücke mit diesen Operatoren ergibt eine Aussage.

Bsp.

$$5 < 10 ; \quad (a+b)^2 = a^2 + 2ab + b^2$$

" $\Rightarrow$ " und " $\Leftrightarrow$ " sind Operatoren, die Aussagen Verknüpfen.

Seien A und B zwei Aussagen

$A \Rightarrow B$ bedeutet A impliziert B

Das heißt: Falls die Aussage A wahr ist, dann ist die Aussage B auch wahr. Allerdings kann die Aussage B wahr sein ohne dass die Aussage A wahr ist.

$A \Leftrightarrow B$ bedeutet A genau dann wenn B

Das heißt: Entweder sind beide Aussagen wahr, oder beide sind falsch. Es ist unmöglich, dass nur eine Aussage wahr ist, während die andere falsch ist.

Beispiel: Wir zeigen, dass $(a+b)^2 = a^2 + 2ab + b^2$

Weg 1:

$$\begin{aligned}(a+b)^2 &= (a+b)(a+b) \\ &= a^2 + ab + ba + b^2 \\ &= a^2 + (ab + ab) + b^2 \\ &= a^2 + 2ab + b^2\end{aligned}$$

In diesem Fall beginnen wir nicht mit einer Aussage.

Wir beginnen mit einem Ausdruck, den wir umformen, bis wir ihn in die gewünschte Form bringen, um unsere Behauptung zu beweisen.

Weg 2:

$$\begin{aligned}(a+b)^2 &= a^2 + 2ab + b^2 \\ \Leftrightarrow (a+b)(a+b) &= a^2 + 2ab + b^2 \\ \Rightarrow a^2 + ab + ba + b^2 &= a^2 + 2ab + b^2 \\ \Leftrightarrow a^2 + (ab + ab) + b^2 &= a^2 + 2ab + b^2 \\ \Leftrightarrow a^2 + 2ab + b^2 &= a^2 + 2ab + b^2\end{aligned}$$

und das gilt.

In diesem Fall beginnen wir mit der Aussage, die wir beweisen wollen. Wir formen diese Aussage in äquivalente Aussagen um, indem wir einen oder beide Ausdrücke in

der Aussage umformen. Wir tun dies, bis wir eine Aussage erhalten, die offensichtlich wahr ist. Der Zweck der Äquivalenzzeichen besteht darin, auszudrücken, dass die erste Aussage genau dann wahr ist, wenn die dazwischenliegenden und die letzte Aussage wahr sind. Am Ende wissen wir zwei Dinge:

1. Die letzte Aussage ist wahr.
2. Aufgrund der Äquivalenzkette wissen wir, dass die erste Aussage genau dann wahr ist, wenn die letzte Aussage wahr ist.

Damit können wir unseren Beweis abschließen.

3. Besprechung von Exercise Sheet 2

Exercise 2.1 Induction for inequalities.

(a) Prove via mathematical induction that for all integers $n \geq 5$,

$$2^n > n^2.$$

In your solution, you should address the base case, the induction hypothesis and the induction step.

Solution:

Base case: $n=5$

$$2^5 = 32 > 25 = 5^2$$

$\Rightarrow$ the statement holds for $n=5$.

Induction Hypothesis: We assume the statement holds for $n=k$,
 $k \in \mathbb{N}$, $k \geq 5$

$$\Rightarrow 2^k > k^2$$

Induction Step: We will show the statement holds for
 $n=k+1$

We will show $2^{k+1} > (k+1)^2$

$$2^{k+1} = 2 \cdot 2^k$$

$$> 2 \cdot k^2 \text{ (from IH we know } 2^k > k^2 \text{)}$$

$$= k^2 + k^2$$

$$\geq k^2 + 5k \text{ (we know } k \geq 5 \text{)}$$

$$= k^2 + 2k + 3k$$

$$\geq k^2 + 2k + 15 \text{ (we know } k \geq 5 \text{)}$$

$$> k^2 + 2k + 1$$

$$= (k+1)^2$$

$\Rightarrow$ We have shown $2^{k+1} > (k+1)^2$

$\Rightarrow$ The statement holds for $n=k+1$

$\Rightarrow$ By the principle of mathematical induction, the statement holds for $\forall n \in \mathbb{N}$, $n \geq 5$

(b) Recall that the Fibonacci numbers are defined by $f_0 = 0$, $f_1 = 1$ and the recursion relation $f_{n+1} = f_n + f_{n-1}$ for all $n \geq 1$. For example, $f_2 = 1$, $f_5 = 5$, $f_{10} = 55$, $f_{15} = 610$.

Prove via mathematical induction that $f_n \geq \frac{1}{3} \cdot 1.5^n$ for $n \geq 1$.

In your solution, you should address the base case, the induction hypothesis and the induction step.

Solution:

Base case: $n=1$

$$f_1 = 1 \geq 0.5 = \frac{1}{3} \cdot 1.5^1 = \frac{1}{3} \cdot 1.5^n$$

$\Rightarrow$ the statement holds for $n=1$

$$n=2$$

$$f_2 = 1 \geq 0,75 = \frac{1}{3} \cdot 1,5^2 = \frac{1}{3} \cdot 1,5^n$$

$\Rightarrow$ the statement holds for $n=2$

Hypothesis: We assume the statement holds for all $n \leq k, k \geq 2$ (*)

$$\Rightarrow f_n \geq \frac{1}{3} \cdot 1,5^n \quad \forall n \leq k$$

Induction step: We will show the statement holds for $n=k+1$

We must show that $f_{k+1} \geq \frac{1}{3} \cdot 1,5^{k+1}$

$$f_{k+1} = f_{k-1} + f_k$$

$$(1) \stackrel{IH}{\geq} \frac{1}{3} \cdot 1,5^{k-1} + \frac{1}{3} \cdot 1,5^k$$

$$= \frac{1}{3} \cdot 1,5^{k-1} (1 + 1,5)$$

$$= \frac{1}{3} \cdot 1,5^{k-1} \cdot 2,5$$

$$> \frac{1}{3} \cdot 1,5^{k-1} \cdot 2,25$$

$$= \frac{1}{3} \cdot 1,5^{k-1} \cdot 1,5^2$$

$$= \frac{1}{3} \cdot 1,5^{k+1}$$

$\Rightarrow$ We have shown $f_{k+1} > \frac{1}{3} \cdot 1,5^{k+1}$

$\Rightarrow$ The statement holds for $n=k+1$

$\Rightarrow$ By the principle of mathematical induction, the statement holds for $\forall n \in \mathbb{N}^+$

(*) Be careful to specify $k \geq 2$ in the induction hypothesis.

We can do that because in the base case we have seen that the statement holds for $n=1$ and $n=2$, so by making $k \geq 2$, we still will have shown the statement holds for $n=1$.

Why $k \geq 2$? See (*) in the induction step. If k was ≤ 1 , then

$f_{k-1} = f_0$ and for $f_0 = 0$ it does not hold $f_0 \geq \frac{1}{3} \cdot 1,5^0$

$\Rightarrow$ we cannot use it to write inequality 1.

If in your solution is the same as mine and you say $k \geq 1$ in the hypothesis, you would need to explain the case where $k=1$ separately in the induction step.

Exercise 2.2 Induction for algorithms (2 bonus points).

We are given a list $a_1, \dots, a_n \in \mathbb{N}$ of $n \geq 1$ distinct integers and our task is to find the maximum element a^* in this list:

- (i) $a^* \in \{a_1, \dots, a_n\}$.
- (ii) $a^* \geq a_i$ for all $i = 1, \dots, n$.

Consider the following recursive algorithm:

Algorithm 1 Recursive maximum

```
if  $n = 1$  then
    return  $a_1$ 
else
    Apply this algorithm recursively and find the maximum element  $a'$  among  $a_1, \dots, a_{n-1}$   $\triangleright n \geq 2$ 
    if  $a' > a_n$  then
        return  $a'$ 
    else
        return  $a_n$   $\triangleright a' \leq a_n$ 
```

Solution:

Let $A(n) :=$ "Algorithm 1 returns the maximum element a^* for any list $a_1, \dots, a_n \in \mathbb{N}$ "

We will show that $A(n)$ holds for every $n \in \mathbb{N}$ by induction.

Base Case: We will show $A(n)$ holds for $n=1$

When $n=1$ the list $a_1, \dots, a_n$ contains a single element, namely a_1 . This means that a_1 trivially is the maximum element of the list.

Now we consider what Algorithm 1 does.

It checks if $n=1$ and this is true

$\Rightarrow$ it returns a_1 and stops executing.

Algorithm 1 returns a_1 as the biggest element of the list

Induction Hypothesis: We assume $A(n)$ holds for $n=k$, $k \in \mathbb{N}$ arbitrary but fixed.

$\Rightarrow A(k) =$ "Algorithm 1 returns the maximum element a^* for any list $a_1, \dots, a_k \in \mathbb{N}$ " holds

Induction Step: We will show $A(n)$ holds for $n=k+1$.

We want to show

$A(k+1) =$ "Algorithm 1 returns the maximum element a^* for any list $a_1, \dots, a_k, a_{k+1} \in \mathbb{N}$ "

We run Algorithm 1 with $n=k+1$, where $k \in \mathbb{N}$

$\Rightarrow n=k+1 \geq 2$

$\Rightarrow$ The algorithm enters the else statement ($n=k+1 \geq 2$)

There the algorithm does a recursive call in order to find the maximum element from the list.

$a_1, \dots, a_k (k=n-1)$

In this recursive call we run Algorithm 1 with $n=k$ and the list $a_1, \dots, a_k$.

From the Induction Hypothesis we know that $A(k)$ holds. This means that the recursive call of Algorithm 1 will indeed return a' , where a' is the biggest element among

$a_1, \dots, a_k (k=n-1)$

$\Rightarrow$ from (ii) we know $a' \geq a_i$ for all $i = 1, \dots, k$

Then the algorithm compares a' and a_n .

If $a' > a_{k+1}$, then we know

$a' > a_{k+1}$ and $a' \geq a_i$ for all $i = 1, \dots, k$

$\Rightarrow a' \geq a_i$ for all $i = 1, \dots, k+1$

$\Rightarrow a'$ is the maximum among $a_1, \dots, a_{k+1}$.

When $a' > a_{k+1}$ Algorithm 1 enters the if statement and correctly returns a' as the biggest element among $a_1, \dots, a_{k+1}$.

Otherwise, when $a' \leq a_{k+1}$, then we know

$a' \leq a_{k+1}$ and $a' \geq a_i$ for all $i = 1, \dots, k$

Since $a_{k+1} \leq a_{k+1}$ we can conclude $a_{k+1} \geq a_i$ for all $i = 1, \dots, k+1$.

$\Rightarrow a_{k+1}$ is the maximum among $a_1, \dots, a_{k+1}$

In this case Algorithm 1 enters the else statement and

correctly returns a_{k+1} as the biggest element among $a_1, \dots, a_{k+1}$. Since the if/else statement covers all possibilities for whether a' or a_{k+1} is bigger and in both cases the algorithm returns the correct result, we have shown $A(k+1)$ holds.

$\Rightarrow$ We have shown $A(n)$ holds for all $n \in \mathbb{N}$.

This means we have shown that the algorithm will return the correct maximum element a^* for any list of numbers $a_1, \dots, a_n$ for any $n \in \mathbb{N}$.

Therefore we have shown that the algorithm will correctly return the maximum element of the list of numbers given in the task.

Exercise 2.3 *O*-notation quiz (2 bonus points).

(a) For all the following functions the variable n ranges over $\mathbb{N}$. Prove or disprove the following statements. Justify your answer using Theorems 1 and/or 2.

(1) $2n^5 + 10n^2 \leq O(\frac{1}{100}n^6)$

(2) $n^{10} + 2n^2 + 7 \leq O(100n^9)$

(3) $e^{1.2n} \leq O(e^n)$

(4)* $n^{\frac{2n+3}{n+1}} \leq O(n^2)$ (This question does not count toward the bonus points.)

(b) Find f and g as in Theorem 1 such that $f \leq O(g)$, but the limit $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist. This proves that the first point of Theorem 1 provides a sufficient, but not a necessary condition for $f \leq O(g)$.

Solution:

a) (1) We will prove the statement using Theorem 1.

$$\lim_{n \rightarrow \infty} \frac{2n^5 + 10n^2}{\frac{1}{100}n^6} = \lim_{n \rightarrow \infty} \frac{200n^5}{n^6} + \frac{1000n^2}{n^6} = \lim_{n \rightarrow \infty} \frac{200}{n} + \frac{1000}{n^4} = 0$$

$\Rightarrow 2n^5 + 10n^2 \leq O(\frac{1}{100}n^6)$ by Th. 1 and the statement is true

2) We will disprove this statement using Theorem 1

$$\lim_{n \rightarrow \infty} \frac{n^{10} + 2n^2 + 7}{100n^9} = \lim_{n \rightarrow \infty} \frac{n^{10}}{100n^9} + \frac{2n^2}{100n^9} + \frac{7}{100n^9} =$$

$$\lim_{n \rightarrow \infty} \frac{n}{100} + \frac{1}{50n^7} + \frac{7}{100n^9} = \lim_{n \rightarrow \infty} \frac{n}{100} = \infty$$

(3) We will disprove the the statement using Theorem 1.

$$\lim_{n \rightarrow \infty} \frac{e^{1.2n}}{e^n} = \lim_{n \rightarrow \infty} e^{1.2n - n} = \lim_{n \rightarrow \infty} e^{0.2n} = \infty$$

$\Rightarrow e^{1.2n} \neq O(e^n)$ By Th.1 and the statement is false

(4) We will prove the statement using Theorem 1.

$$\lim_{n \rightarrow \infty} \frac{n^{\frac{2n+3}{n+1}}}{n^2} = \lim_{n \rightarrow \infty} \frac{e^{\ln n \cdot \frac{2n+3}{n+1}}}{e^{\ln n \cdot 2}} = \lim_{n \rightarrow \infty} \frac{e^{\frac{2n+3}{n+1} \cdot \ln n}}{e^{2 \ln n}} =$$

$$\lim_{n \rightarrow \infty} e^{\frac{2n+3}{n+1} \cdot \ln n - 2 \ln n} = \lim_{n \rightarrow \infty} e^{\ln n \left(\frac{2n+3}{n+1} - 2 \right)} =$$

$$e^{\lim_{n \rightarrow \infty} \ln n \left(\frac{2n+3}{n+1} - 2 \right)}$$

$$\lim_{n \rightarrow \infty} \ln n \left(\frac{2n+3}{n+1} - 2 \right) = \lim_{n \rightarrow \infty} \ln n \left(\frac{2(n+1)+1}{n+1} - 2 \right) =$$

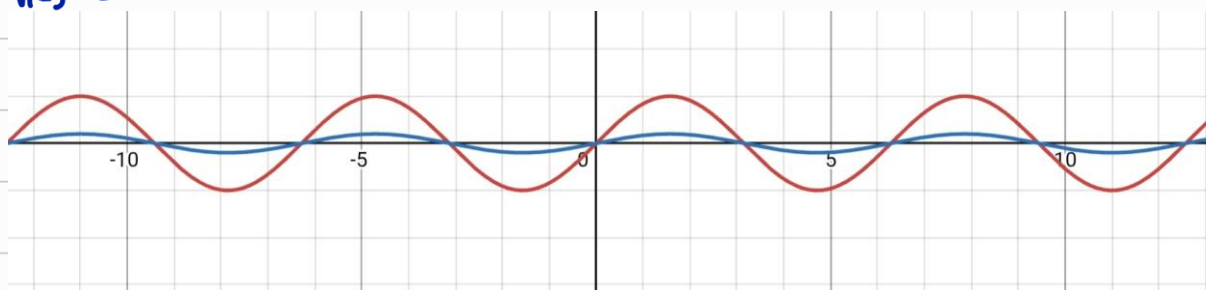
$$\lim_{n \rightarrow \infty} \ln n \left(2 + \frac{1}{n+1} - 2 \right) = \lim_{n \rightarrow \infty} \ln n \cdot \frac{1}{n+1} = \lim_{n \rightarrow \infty} \frac{\ln n}{n+1} = 0$$

$$\Rightarrow \lim_{n \rightarrow \infty} \frac{n^{\frac{2n+3}{n+1}}}{n^2} = e^{\lim_{n \rightarrow \infty} \ln n \left(\frac{2n+3}{n+1} - 2 \right)} = e^0 = 1$$

$\Rightarrow n^{\frac{2n+3}{n+1}} \leq O(n^2)$ By Th 1 and the statement is true

(6) We define $f(x) := \sin(x)$, $g(x) = 5$.

$\lim_{n \rightarrow \infty} \frac{\sin(x)}{5}$ does not exist



$$f(x) = \sin(x)$$

$$g(x) = \frac{\sin(x)}{5}$$

Exercise 2.4 Asymptotic growth of $\sum_{i=1}^n \frac{1}{i}$ (1 point).

The goal of this exercise is to show that the sum $\sum_{i=1}^n \frac{1}{i}$ behaves, up to constant factors, as $\log(n)$ when n is large. Formally, we will show $\sum_{i=1}^n \frac{1}{i} \leq O(\log n)$ and $\log n \leq O(\sum_{i=1}^n \frac{1}{i})$ as functions from $\mathbb{N}_{\geq 2}$ to $\mathbb{R}^+$.

For parts (a) to (c) we assume that $n = 2^k$ is a power of 2 for $k \in \mathbb{N}_0 = \mathbb{N} \cup \{0\}$. We will generalise the result to arbitrary $n \in \mathbb{N}$ in part (d). For $j \in \mathbb{N}$, define

$$S_j = \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{i}.$$

(a) For any $j \in \mathbb{N}$, prove that $S_j \leq 1$.

Hint: Find a common upper bound for all terms in the sum and count the number of terms.

Solution:

$$a) S_j = \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{i}, \quad j \text{ is arbitrary}$$

$$\Rightarrow i \geq 2^{j-1}+1$$

$$\Rightarrow \frac{1}{i} \leq \frac{1}{2^{j-1}+1}$$

Additionally, we know that i iterates over all integers between $2^{j-1}+1$ and 2^j , inclusive

$\Rightarrow$ the sum $\sum_{i=2^{j-1}+1}^{2^j} \frac{1}{i}$ has:

$$2^j - (2^{j-1}+1) + 1 = 2^j - 2^{j-1} = 2 \cdot 2^{j-1} - 2^{j-1} = 2^{j-1} \text{ terms}$$

$\Rightarrow$ We know that the sum has 2^{j-1} terms and for each term $\frac{1}{i}$ holds $\frac{1}{i} \leq \frac{1}{2^{j-1}+1}$

$$\Rightarrow S_j = \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{i} \leq \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{2^{j-1}+1} = 2^{j-1} \cdot \frac{1}{2^{j-1}+1} = \frac{2^{j-1}}{2^{j-1}+1} < 1$$

$\Rightarrow S_j \leq 1$ and since j is arbitrary,

$$S_j \leq 1 \quad \forall j \in \mathbb{N}$$

(b) For any $j \in \mathbb{N}$, prove that $S_j \geq \frac{1}{2}$.

$$b) S_j = \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{i}, \quad j \text{ is arbitrary}$$

$$\Rightarrow i \leq 2^j$$

$$\Rightarrow \frac{1}{i} \geq \frac{1}{2^j}$$

$$\Rightarrow S_j = \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{i} \geq \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{2^j}$$

The sum $\sum_{i=2^{j-1}}^{2^j}$ has $2^j - (2^{j-1} + 1) + 1 = 2 \cdot 2^{j-1} - 2^{j-1} = 2^{j-1}$ terms
 $\Rightarrow S_j = \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{i} \geq \sum_{i=2^{j-1}+1}^{2^j} \frac{1}{2^j} = 2^{j-1} \cdot \frac{1}{2^j} = \frac{1}{2}$

$\Rightarrow S_j \geq \frac{1}{2}$ and since j is arbitrary $S_j \geq \frac{1}{2} \forall j \in \mathbb{N}$

c) Let $k \in \mathbb{N}_0$ be arbitrary

$$\begin{aligned} \sum_{i=1}^{2^k} \frac{1}{i} &= 1 + \sum_{i=2^0+1}^{2^1} \frac{1}{i} + \sum_{i=2^1+1}^{2^2} \frac{1}{i} + \dots + \sum_{i=2^{k-1}+1}^{2^k} \frac{1}{i} \\ &= 1 + \sum_{j=1}^k S_j \end{aligned}$$

From part a) we know that $S_j \leq 1 \forall j \in \mathbb{N}$

$$\Rightarrow \sum_{i=1}^{2^k} \frac{1}{i} = 1 + \sum_{j=1}^k S_j \leq 1 + k \cdot 1 = 1 + k$$

$\Rightarrow \sum_{i=1}^{2^k} \frac{1}{i} \leq k+1$ and since k is arbitrary $\sum_{i=1}^{2^k} \frac{1}{i} \leq k+1 \forall k \in \mathbb{N}_0$

From part b) we know that $S_j \geq \frac{1}{2} \forall j \in \mathbb{N}$

$$\Rightarrow \sum_{i=1}^{2^k} \frac{1}{i} = 1 + \sum_{j=1}^k S_j \geq 1 + k \cdot \frac{1}{2} > \frac{k+1}{2}$$

$\Rightarrow \sum_{i=1}^{2^k} \frac{1}{i} \geq \frac{k+1}{2}$ and since k is arbitrary $\sum_{i=1}^{2^k} \frac{1}{i} \geq \frac{k+1}{2} \forall k \in \mathbb{N}_0$

d) Let n be arbitrary and $k_1 = \lceil \log_2 n \rceil$, $k_2 = \lfloor \log_2 n \rfloor$

$$\Rightarrow \sum_{i=1}^n \frac{1}{i} \leq \sum_{i=1}^{2^{k_1}} \frac{1}{i} \quad \text{and} \quad \sum_{i=1}^n \frac{1}{i} \geq \sum_{i=1}^{2^{k_2}} \frac{1}{i}$$

From part c) we know that $\sum_{i=1}^{2^k} \frac{1}{i} \leq k+1 \forall k \in \mathbb{N}_0$, including k_1

$$\Rightarrow \sum_{i=1}^{2^{k_1}} \frac{1}{i} \leq k_1 + 1$$

We know $k_1 = \lceil \log_2 n \rceil \Rightarrow k_1 \leq \log_2 n + 1$

$$\Rightarrow \sum_{i=1}^n \frac{1}{i} \leq \sum_{i=1}^{2^{k_1}} \frac{1}{i} \leq k_1 + 1 \leq \log_2 n + 1 + 1$$

$\Rightarrow \sum_{i=1}^n \frac{1}{i} \leq \log_2 n + 2$ and since n is arbitrary,

$$\sum_{i=1}^n \frac{1}{i} \leq \log_2 n + 2 \quad \forall n \in \mathbb{N}$$

From part c) we know that $\sum_{i=1}^{2^{k_2}} \frac{1}{i} \geq \frac{k_2+1}{2} \quad \forall k_2 \in \mathbb{N}_0$, including k_2
 $\Rightarrow \sum_{i=1}^{2^{k_2}} \frac{1}{i} \geq \frac{k_2+1}{2}$

We know $k_2 = \lfloor \log_2 n \rfloor \Rightarrow k_2 \geq \log_2 n - 1$

$$\Rightarrow \sum_{i=1}^n \frac{1}{i} \geq \sum_{i=1}^{2^{k_2}} \frac{1}{i} \geq \frac{k_2+1}{2} \geq \frac{\log_2 n - 1 + 1}{2} = \frac{\log_2 n}{2}$$

$\Rightarrow \sum_{i=1}^n \frac{1}{i} \geq \frac{\log_2 n}{2}$ and since n is arbitrary
 $\sum_{i=1}^n \frac{1}{i} \geq \frac{\log_2 n}{2} \quad \forall n \in \mathbb{N}$

Exercise 2.5 Asymptotic growth of $\ln(n!)$.

Recall that the factorial of a positive integer n is defined as $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$. For the following functions n ranges over $\mathbb{N}_{\geq 2}$.

(a) Show that $\ln(n!) \leq O(n \ln n)$.

Hint: You can use the fact that $n! \leq n^n$ for $n \in \mathbb{N}_{\geq 2}$ without proof.

(b) Show that $n \ln n \leq O(\ln(n!))$.

Hint: You can use the fact that $\left(\frac{n}{2}\right)^{\frac{n}{2}} \leq n!$ for $n \in \mathbb{N}_{\geq 2}$ without proof.

Solution:

a) We know that $\ln n$ is monotonically increasing for $n \in \mathbb{N}_{\geq 2}$

$\Rightarrow$ since $n! \leq n^n \quad \forall n \in \mathbb{N}_{\geq 2}$, then

$$\ln(n!) \leq \ln(n^n) \quad \forall n \in \mathbb{N}_{\geq 2}$$

$$\Rightarrow \ln(n!) \leq n \ln(n) \quad \forall n \in \mathbb{N}_{\geq 2}$$

$\Rightarrow \exists C > 0$ (namely $C=1$) $\forall n \in \mathbb{N}_{\geq 2}$, such that $\ln(n!) \leq C \cdot n \ln(n)$

$\Rightarrow \ln(n!) \leq O(n \ln(n))$ by Definition 1.

b) We know that $\left(\frac{n}{2}\right)^{\frac{n}{2}} \leq n! \quad \forall n \in \mathbb{N}_{\geq 2}$ and $\ln n$ is monotonically increasing for $n \in \mathbb{N}_{\geq 2}$

$$\Rightarrow \ln\left(\left(\frac{n}{2}\right)^{\frac{n}{2}}\right) \leq \ln(n!)$$

$$\Rightarrow \frac{n}{2} \ln\left(\frac{n}{2}\right) \leq \ln(n!)$$

$$\Rightarrow n \ln\left(\frac{n}{2}\right) \leq 2 \ln(n!)$$

$$\Rightarrow n(\ln n - \ln 2) \leq 2 \ln(n!)$$

$$\Rightarrow n \ln n - n \ln 2 \leq 2 \ln(n!)$$

$$\Rightarrow n \ln n \leq 2 \ln(n!) + n \ln 2$$

$$n \ln 2 = \sum_{i=1}^n \ln 2 = \ln 2 + \sum_{i=2}^n \ln 2 \leq \ln 2 + \sum_{i=2}^n \ln i = \ln 2 + \ln(n!)$$

$$\Rightarrow n \ln 2 \leq \ln 2 + \ln(n!)$$

$$\text{and } \ln 2 \leq \ln(n!) \quad (n \in \mathbb{N}_{\geq 2})$$

$$\Rightarrow n \ln 2 \leq 2 \ln(n!)$$

$$\Rightarrow n \ln n \leq 2 \ln(n!) + n \ln 2 \leq 2 \ln(n!) + 2 \ln(n!) = 4 \ln(n!)$$

$$\Rightarrow n \ln n \leq 4 \ln(n!)$$

$$\Rightarrow \exists C > 0 \text{ (namely } C=4) \forall n \in \mathbb{N}_{\geq 2}, \text{ such that } n \ln n \leq C \cdot \ln(n!)$$

$$\Rightarrow n \ln n \leq O(\ln(n!))$$

4. Theory - recap

Maximum Subarray Sum

gegeben: $a_1, \dots, a_n \in \mathbb{Z}$ (ganze Zahlen)

gesucht: maximale Teilsumme $S_{i,j}$ mit $i, j \in \{1, \dots, n\}$ und $i \leq j$

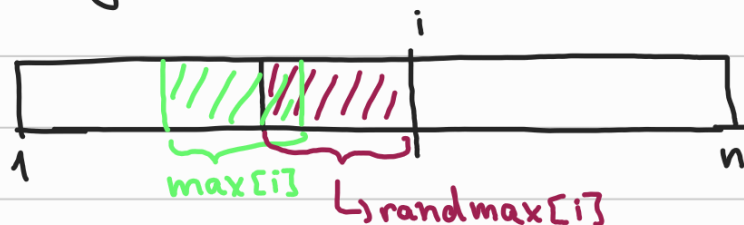
$S_{i,j} = a_i + \dots + a_j$ und $a_i, \dots, a_j$ sind aufeinanderfolgende Elemente aus $a_1, \dots, a_n$.

Falls $a_k < 0$ für alle $k \in \{1, \dots, n\}$, dann $S = 0$

(die leere Menge hat Summe 0)

Elementare Operation: Addition

Algorithmus 4 (induktiv von links nach rechts)



Sei $i \in \{1, \dots, n\}$ beliebig. Wir betrachten nur die Elemente $a_1, a_2, \dots, a_i$ und wir definieren zwei Summen: $\text{randmax}[i]$ und $\text{max}[i]$.

- $\text{randmax}[i]$: die größte Subarray Sum, deren letzte

Element a_i ist.

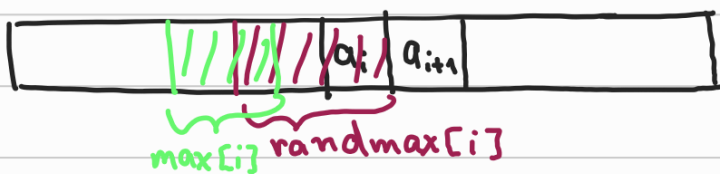
- $\max[i]$: die größte Subarray Sum, deren letzte Element Index kleiner oder gleich i hat.

$$\max[i] = \max_{1 \leq j \leq i} \text{randmax}[j]$$

Wir wissen, dass $\max[i] \geq \text{randmax}[i]$ ist, da wir wissen, dass $\max[i]$ die größte aller Subarray Sums ist, die an einem Index kleiner oder gleich i enden, und das schließt die Summe $\text{randmax}[i]$ ein.

Wir wissen auch, dass mindestens ein j existiert, sodass $\max[j] = \text{randmax}[j]$. Das liegt daran, dass jede Subarray Sum an irgendeinem Index enden muss. Wir nehmen an, dass der Index, an dem die größte Subarray Sum endet, j ist. Da die größte Summe bei j endet, bedeutet dies, dass die Summe, die $\max[j]$ enthält, ebenfalls bei j endet, und das bedeutet, dass $\max[j] = \text{randmax}[j]$ ist.

Nun nehmen wir an, dass wir $\text{randmax}[i]$ und $\max[i]$ kennen. Wir wollen $\text{randmax}[i+1]$ und $\max[i+1]$ berechnen



Fall 1: $a_{i+1} \geq 0$

Fall 1.1. $\text{randmax}[i] > 0$

Dann $\text{randmax}[i+1] = \text{randmax}[i] + a_{i+1}$ und
 $\max[i+1] = \max(\max[i], \text{randmax}[i+1])$

Fall 1.2. $\text{randmax}[i] \leq 0$

Dann $\text{randmax}[i+1] = a_{i+1}$ und

$$\max[i+1] = \max(\max[i], a_{i+1}) = \max(\max[i], \text{randmax}[i])$$

Fall 2 $a_{i+1} < 0$

Fall 2.1 $\text{randmax}[i] > 0$

Dann $\text{randmax}[i+1] = \text{randmax}[i] + a_{i+1}$ und
 $\max[i+1] = \max[i]$ Warum?

Fall 2.2 $\text{randmax}[i] \leq 0$

Dann $\text{randmax}[i+1] = a_{i+1}$ und $\rightarrow$ Warum?

$$\max[i+1] = \max(\max[i], a_{i+1}) = \max(\max[i], \text{randmax}[i])$$

Am Ende enthält $\max[n]$ die größte Subarray Sum, das in ein index kleiner oder gleich n endet.

Das bedeutet, dass $\max[n]$ die größte Subarray Sum enthält. Am Anfang stellen wir $\max[i] = 0 \forall i \in \{1, \dots, n\}$.

Asymptotische Notation

Asymptotic Notation

The following two definitions are closely related to the O -notation and are also useful in the running time analysis of algorithms. Let N be again a set of possible inputs (e.g. $N = \{n_0, n_0 + 1, \dots\}$ for $n_0 \in \mathbb{N}$).

Definition 1 (Ω -Notation). For $f : N \rightarrow \mathbb{R}^+$,

$$\begin{aligned} \Omega(f) &:= \{g : N \rightarrow \mathbb{R}^+ \mid f \leq O(g)\} \\ &= \{g : N \rightarrow \mathbb{R}^+ \mid \text{there exists } C > 0 \text{ such that for all } n \in N \ g(n) \geq C \cdot f(n)\}. \end{aligned}$$

We write $g \geq \Omega(f)$ instead of $g \in \Omega(f)$.

Ω -notation is a way to lower bound the asymptotic behavior of a function, like the runtime of an algorithm.

$\Omega(f)$ ist die Menge, die alle Funktionen enthält, die asymptotisch schneller wachsen als f oder asymptotisch gleich wie f wachsen.

Wir wissen:

falls $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \infty$, dann $g(n)$ wächst asymptotisch schneller als $f(n)$

$$\Rightarrow g(n) \in \Omega(f) \quad (g(n) \geq \Omega(f))$$

falls $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = C \in \mathbb{R}^+$, dann $g(n), f(n)$ wachsen asymptotisch gleich

$$\Rightarrow g(n) \in \Omega(f) \quad (g(n) \geq \Omega(f))$$

Definition 2 (Θ -Notation). For $f : N \rightarrow \mathbb{R}^+$,

$$\begin{aligned} \Theta(f) &:= \{g : N \rightarrow \mathbb{R}^+ \mid g \leq O(f) \text{ and } f \leq O(g)\} \\ &= \{g : N \rightarrow \mathbb{R}^+ \mid \text{there exist } C_1, C_2 > 0 \text{ s.t. for all } n \in N \ C_1 \cdot f(n) \leq g(n) \leq C_2 \cdot f(n)\}. \end{aligned}$$

We write $g = \Theta(f)$ instead of $g \in \Theta(f)$.

Θ -notation is a way to simultaneously upper and lower bound the asymptotic behavior of a function.

In other words, for two functions $f, g : N \rightarrow \mathbb{R}^+$ we have

$$g \geq \Omega(f) \Leftrightarrow f \leq O(g)$$

and

$$g = \Theta(f) \Leftrightarrow g \leq O(f) \text{ and } f \leq O(g).$$

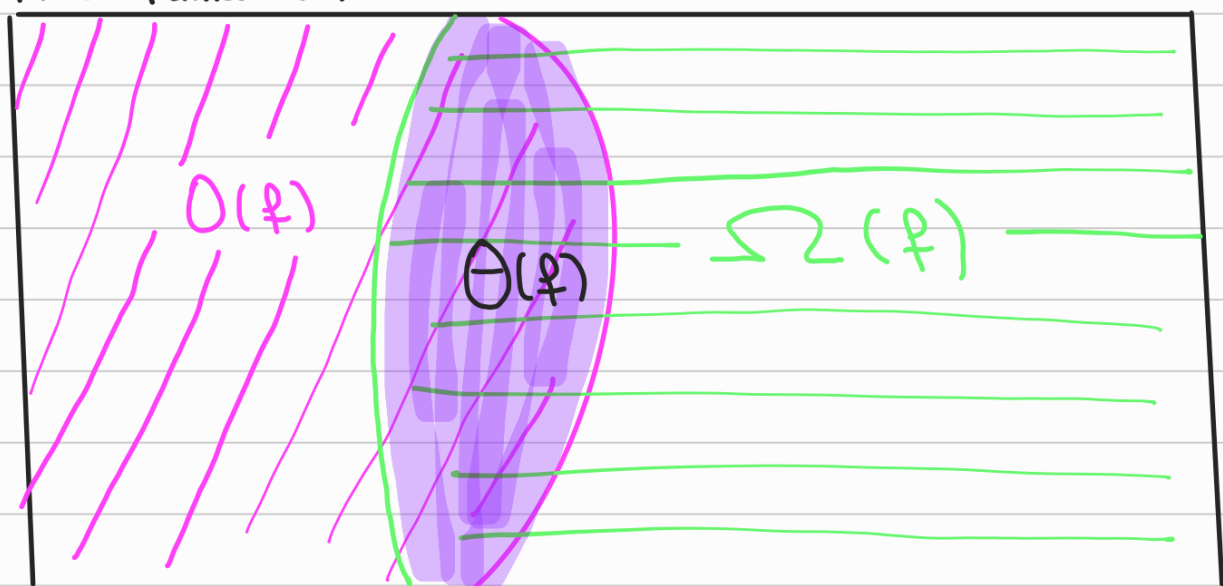
We can restate Theorem 1 from exercise sheet 2 as follows.

$\Theta(f)$ ist die Menge, die alle Funktionen enthält, die asymptotisch genau so schnell wie f wachsen.

Wir wissen:

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = C \in \mathbb{R}^+ &\Rightarrow f(n) \text{ und } g(n) \text{ wachsen asymptotisch gleich} \\ &\Rightarrow g(n) \in \Theta(f) \quad (g(n) = \Theta(f)) \end{aligned}$$

Alle Funktionen



5. In-Class Aufgaben

Übung 1

	Aussage	Wahr	Falsch
1.	$\Theta(f) \subseteq O(f)$		
2.	$\Theta(f) \subseteq \Omega(f)$		
3.	$O(f) \subseteq \Omega(f)$		
4.	$O(f) \cap \Omega(f) = \emptyset$		
5.	$O(f) \cap \Omega(f) = \Theta(f)$		
6.	$\Theta(f) \cap \Omega(f) = \Theta(f)$		

Übung 2

	Claim	True	False
1)	$f(n) = n^3, g(n) = 2^n, f \leq O(g)$		
2)	$f(n) = 2^n, g(n) = 3^n, f \leq O(g)$		
3)	$f(n) = \sqrt{n}, g(n) = \ln^2 n, f \leq O(g)$		
4)	$f(n) = n \log(n), g(n) = n \cdot (\log n)^2, f \leq O(g)$		
5)	$f(n) = n^{\ln(n)}, g(n) = 2^n, f \leq O(g)$		

Lösung:

	Claim	True	False
1)	$f(n) = n^3, g(n) = 2^n, f \leq O(g)$		
2)	$f(n) = 2^n, g(n) = 3^n, f \leq O(g)$		
3)	$f(n) = \sqrt{n}, g(n) = \ln^2 n, f \leq O(g)$		
4)	$f(n) = n \log(n), g(n) = n \cdot (\log n)^2, f \leq O(g)$		
5)	$f(n) = n^{\ln(n)}, g(n) = 2^n, f \leq O(g)$		

Übung 3 Aus Exercise Sheet 2, HS 25

- (b) Let x be any real number. Prove via mathematical induction that for every positive integer n , we have

$$(1+x)^n = \sum_{i=0}^n \binom{n}{i} x^i,$$

where

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}.$$

Recall that the factorial of a positive integer n is defined as $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$. We use a standard convention $0! = 1$, so $\binom{n}{0} = \binom{n}{n} = 1$ for every positive integer n .

In your solution, you should address the base case, the induction hypothesis and the induction step.

Hint: You can use the following fact without proof: for every $1 \leq i \leq n$,

$$\binom{n}{i} + \binom{n}{i-1} = \binom{n+1}{i}.$$

Lösung:

